

Facultad de Ingeniería – Ingeniería Informática - UNSTA

Programación II – 3° Trabajo Práctico

Alumno: Alan Coronel CI180-126

Profesor: Ing. Tulio Ruesjas Martín

Fecha de entrega: 04 de mayo de 2026

Ejercicio 1 – Sistema de Flota de Transporte

Descripción: Se modeló una flota de vehículos compuesta por furgonetas y motocicletas. Se utilizó una clase abstracta Vehiculo como base con los atributos comunes marca, modelo y tarifaBase. Las clases Furgoneta y Motocicleta extienden Vehiculo, agregando sus atributos propios (capacidad en kg y cilindrada respectivamente).

Decisiones de diseño: Se implementó la interfaz TieneCaracteristica con el método getCaracteristica() para exponer la característica técnica de cada vehículo de forma polimórfica. El método mostrarInfo() se declaró abstracto en la clase padre y fue sobrescrito en cada hijo. La clase Flota encapsula el ArrayList y el método emitirReporte() recorre la colección aplicando polimorfismo.

Observaciones: La palabra 'actualmente' en el enunciado indica extensibilidad futura. El diseño permite agregar nuevos tipos de vehículos sin modificar el código existente.

Ejercicio 2 – Gestor de Suscripciones de Streaming

Descripción: Se modeló un sistema de suscripciones con tres planes: PlanBasico, PlanFamiliar y PlanPremium. La clase abstracta Suscripcion agrupa los datos comunes (correo, número de cliente y costo base). El número de cliente se genera automáticamente con Random. El costo base se declaró como constante en la clase padre.

Decisiones de diseño: El método abstracto calcularCostoMensual() obliga a cada subclase a implementar su propia lógica de facturación. PlanBasico retorna el costo base directamente. PlanFamiliar agrega un recargo por perfiles adicionales. PlanPremium suma un cargo fijo por las características premium. La clase MotorFacturacion recorre la lista y acumula el total usando polimorfismo.

Observaciones: No se utilizó interfaz en este ejercicio ya que todos los comportamientos pertenecen a la misma jerarquía de Suscripcion.

Ejercicio 3 – Pasarela de Pagos E-commerce

Descripción: Se implementaron tres procesadores de pago: TarjetaCredito, PayPal y Criptomoneda. Cada uno tiene una lógica de procesamiento completamente distinta e independiente de las demás.

Decisiones de diseño: Se utilizó exclusivamente una interfaz ProcesarPago con el método procesarPago(double monto) ya que las tres clases no comparten ninguna relación de herencia entre sí. Este es el caso de uso natural de la interfaz: un contrato que cruza clases sin jerarquía común. La clase CarritoCompra encapsula la lista y el motor de procesamiento.

Observaciones: No se utilizó clase abstracta porque no existen datos comunes entre los procesadores. Agregar un nuevo método de pago solo requiere crear una nueva clase que implemente la interfaz, sin modificar nada del sistema existente.

Ejercicio 4 – Ecosistema de Dispositivos Inteligentes

Descripción: Se modelaron tres dispositivos: CamaraSeguridad, TermostatoInteligente y Smartphone. Cada uno posee capacidades específicas: tomar fotos, conectarse a WiFi, o ambas en el caso del Smartphone.

Decisiones de diseño: Se definieron dos interfaces: TomarFoto y ConectarWifi. El Smartphone implementa ambas, demostrando herencia múltiple de comportamiento via interfaces. El ControladorCentral utiliza instanceof y cast para identificar qué dispositivos poseen cada capacidad e invocar el método correspondiente.

Observaciones: No se utilizó clase abstracta ya que el enunciado no especifica atributos comunes entre los dispositivos. La lista es de tipo Object para poder contener cualquier dispositivo independientemente de su tipo.

Ejercicio 5 – Motor de Videojuego de Rol

Descripción: Se diseñó la arquitectura completa de un motor de videojuego con jerarquía multinivel. EntidadEspacial es la raíz con coordenadas X e Y y el método abstracto dibujar(). SerVivo extiende EntidadEspacial agregando puntos de vida, recibirDanio() y estaVivo(). PersonajeJugable y MonstruoHostil extienden SerVivo con sus propias lógicas de interacción. Las clases concretas son GuerreroHumano, MagoElfo, Orco y Dragon.

Decisiones de diseño: Se utilizó la interfaz LanzarHechizo para la capacidad mágica, ya que cruza la jerarquía: MagoElfo (jugable) y Dragon (hostil) la implementan, mientras que GuerreroHumano y Orco no. El motor principal usa tres listas con instanceof y cast para: actualizar coordenadas de todas las entidades, calcular supervivencia de los seres vivos, y lanzar hechizos solo a los que implementan la interfaz.

Observaciones: Este ejercicio es el más completo del TP ya que combina herencia multinivel, clases abstractas en múltiples niveles e interfaz cruzando jerarquías. La arquitectura permite extender el juego con nuevas entidades sin modificar el motor existente.

Dificultades encontradas

Lo que más tiempo llevó comprender fue el funcionamiento de las listas (ArrayList), el recorrido con foreach y el uso combinado de instanceof y cast.

ArrayList: Entender que el tipo entre <> define qué objetos acepta la lista, y que al ser de tipo padre (por ejemplo List<Vehiculo>) puede contener cualquier objeto de una subclase.

foreach: Comprender que al recorrer la lista con for-each, Java ejecuta automáticamente el método del tipo real del objeto y no del tipo declarado en la lista. Ese es el momento donde el polimorfismo se hace visible.

instanceof y cast: Fue el concepto más complejo del TP. Al usar listas genéricas (List<Object> o List<EntidadEspacial>), Java pierde el tipo específico del objeto. El instanceof permite verificar si un objeto implementa una interfaz o pertenece a una clase antes de hacer el cast, que es la conversión temporal de tipo necesaria para poder invocar métodos específicos. Sin la verificación previa con instanceof, el cast puede lanzar una excepción en tiempo de ejecución.